

RAG Chatbot

Project Documentation

1. Project Overview

RAG Chatbot is a document-aware conversational AI application. Users upload a plain-text document, then ask questions about it. The system retrieves the most relevant passages from the document using vector search (RAG) and passes them to Google Gemini to generate accurate, grounded answers. Conversation history is tracked per session so the bot understands follow-up questions.

Key capabilities:

- Upload any plain-text (.txt) document
- Ask natural language questions — answers come from your document
- Session-aware memory — follow-up questions are understood in context
- Fully containerised — runs anywhere with Docker
- API key stored in .env — never exposed in commands

2. Architecture

2.1 High-Level Flow

User uploads .txt → Text is chunked → Chunks embedded with SentenceTransformer → Stored in FAISS index (saved to disk)

User asks question → Question embedded → Top-3 matching chunks retrieved → Chunks + session history sent to Gemini → Answer returned

2.2 Project Structure

Path	Purpose
app/main.py	Fast API application — routes, Gemini client, session memory
app/rag.py	RAG engine — chunking, embedding, FAISS index, disk persistence

Path	Purpose
templates/index.html	Frontend HTML — upload form + chat UI
static/style.css	Frontend styles
requirements.txt	Python dependencies
Dockerfile	Container build instructions
.env	API key (never committed to git)
data/	Persisted FAISS index and document chunks (auto-created)

3. Modules & Libraries

3.1 FastAPI

Web framework used to expose three HTTP endpoints: GET / (UI), POST /upload (document ingestion), POST /ask (question answering). Chosen for its async support, automatic request validation via Pydantic, and built-in OpenAPI docs.

3.2 Uvicorn

ASGI server that runs the FastAPI application. Handles async HTTP connections. Configured to listen on 0.0.0.0:8000 inside the container.

3.3 Jinja2 + python-multipart

Jinja2 is the HTML templating engine used by FastAPI to render index.html. python-multipart enables the /upload endpoint to receive file form-data from the browser.

3.4 sentence-transformers

Provides the all-MiniLM-L6-v2 embedding model — a lightweight 384-dimension transformer that converts text chunks and queries into dense vectors. Used in rag.py for both indexing documents and searching at query time. Runs fully locally, no API call needed.

3.5 FAISS (faiss-cpu)

Facebook AI Similarity Search — an efficient vector similarity library. Stores document chunk embeddings in an IndexFlatL2 index (exact L2 distance search). The index is written to data/faiss.index and reloaded on startup, so documents persist across container restarts.

3.6 google-genai

The official Google Generative AI Python SDK (new version). Used to call the Gemini 2.0 Flash model via `genai.Client.models.generate_content()`. The API key is loaded from the GEMINI_API_KEY environment variable set via .env.

3.7 python-dotenv

Loads key=value pairs from the .env file into environment variables at startup (load_dotenv()). This means the Gemini API key never needs to be passed on the command line — it is read automatically from the file.

3.8 Session Memory (built-in)

Implemented as a plain Python defaultdict(list) in main.py — no external service required. Each session_id maps to a list of {role, content} turns. The last 10 turns are included in every Gemini prompt so the model understands follow-up questions.

4. API Endpoints

Method	Path	Description	Request	Response
GET	/	Serves the chat web UI	None	HTML page
POST	/upload	Ingests a .txt document into the FAISS index	multipart/form-data file=<file>	{"msg": "...uploaded..."}
POST	/ask	Answers a question using RAG + session memory	{"session_id": "x", "question": "y"}	{"answer": "..."}

5. Configuration

5.1 Environment Variables (.env)

Variable	Required	Description
GEMINI_API_KEY	Yes	Your Google AI Studio API key for Gemini 2.5 Flash

5.2 RAG Parameters (app/rag.py)

Parameter	Default	Description
chunk_size	200 words	Size of each text chunk passed to the embedder
k (query_docs)	3	Number of top matching chunks retrieved per query

Parameter	Default	Description
model	all-MiniLM-L6-v2	SentenceTransformer model used for embeddings

5.3 Session Memory (app/main.py)

Parameter	Default	Description
history[-10:]	Last 10 turns	Number of conversation turns included in each prompt

6. Data Persistence

The FAISS vector index and raw document chunks are saved to disk inside the container at `data/faiss.index` and `data/documents.pkl`. When running with Docker, the `-v` flag mounts this directory to your host machine so data survives container restarts:

```
-v ${PWD}/data:/app/data
```

Session memory (conversation history) is in-memory only and is cleared when the container stops. This is by design — sessions are per-run.

7. Known Limitations

- Only plain `.txt` files are supported — PDF, DOCX, etc. are not yet handled
- Session memory is in-memory — cleared on container restart
- FAISS index holds only the most recently uploaded document — re-uploading replaces it
- Gemini free tier has rate limits — errors will occur if quota is exceeded
- The HF Hub warning about unauthenticated requests is harmless — model weights are cached after first download